PROJECT REPORT

FOR

Foundations of Artificial Intelligence

SCRIBE AI: MEDICAL SOAP NOTES GENERATOR

A PROJECT REPORT

Submitted by

240701363 – NISHANI B

240701354 – NEHA RN



October, 2026

BONAFIDE CERTIFICATE

Certified that this project report “SCRIBE AI: MEDICAL SOAP NOTES GENERATOR” is the bonafide work of “NISHANI B (240701363) and NEHA RN (240701354)” who carried out the project work under my supervision.

SIGNATURE OF THE FACULTY INCHARGE

Submitted for the Practical Examination held on _____

SIGNATURE OF THE INTERNAL EXAMINER

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	ii
1	**INTRODUCTION**	1
1.1	BACKGROUND	1
1.2	PROBLEM STATEMENT	1
1.3	OBJECTIVES	2
1.4	SCOPE AND LIMITATIONS	2
1.5	ORGANISATION OF THE REPORT	2
2	**LITERATURE REVIEW**	3
2.1	CLINICAL DOCUMENTATION BURDEN	3
2.2	THE SOAP NOTE FORMAT	3
2.3	AUTOMATIC SPEECH RECOGNITION	3
2.4	CLINICAL NLP AND LANGUAGE MODELS	4
2.5	HUMAN OVERSIGHT	4
2.6	SUMMARY AND RESEARCH GAP	4
3	**PROPOSED SYSTEM**	6
3.1	ARCHITECTURE DIAGRAM	6
3.2	SYSTEM WORKFLOW	7
3.3	TECHNOLOGY STACK	7
3.4	API DESIGN	8
3.5	DESIGN CONSIDERATIONS	8
4	**MODULES DESCRIPTION**	9
4.1	MODULE 1: AUDIO CAPTURE AND UPLOAD	9
4.2	MODULE 2: SPEECH-TO-TEXT	9
4.3	MODULE 3: CLINICAL NLP AND EXTRACTION	9
4.4	MODULE 4: SOAP NOTE GENERATION	10
4.5	MODULE 5: REVIEW AND EDITING	10
4.6	MODULE 6: STORAGE AND EXPORT	10
4.7	MODULE 7: AUTHENTICATION AND AUTHORISATION	10
5	**IMPLEMENTATION AND RESULTS**	11
5.1	EXPERIMENTAL SETUP	11
5.2	RESULTS	12
5.3	DISCUSSION	13
6	**CONCLUSION AND FUTURE WORK**	14
6.1	CONCLUSION	14
6.2	FUTURE WORK	14

7 REFERENCES 15

ABSTRACT

Scribe AI is a clinical documentation assistant designed to reduce the manual effort involved in preparing consultation notes. The proposed workflow accepts recorded doctor–patient conversations, converts speech into text, and applies natural language processing to organise relevant information into the four sections of a SOAP note: Subjective, Objective, Assessment and Plan. The generated note is presented for clinician review and editing before it is saved or exported. The system combines a React web interface, a FastAPI backend, speech recognition, NLP-based information extraction and persistent storage. It is intended to support documentation efficiency, not to replace clinical judgement or independently make diagnoses. This report presents the problem, a review of related work, the proposed architecture and modules, the implementation approach and an evaluation plan covering transcription quality, latency, note completeness and review effort.

CHAPTER 1
INTRODUCTION

1.1 BACKGROUND

Clinical documentation is a core part of healthcare delivery. After every consultation, a clinician must record what the patient reported, what was observed or measured, what the clinician concluded, and what will be done next. One of the most widely used structures for this record is the SOAP note, which organises a patient encounter into four sections: Subjective, Objective, Assessment and Plan (Podder, Lew and Ghassemzadeh, 2023). The format gives every consultation a consistent shape, which makes notes easier to read, share and review.

Producing such notes by hand is time-consuming. In a time-and-motion study of physicians in ambulatory practice, Sinsky et al. (2016) found that physicians spent a large share of their working day on electronic health record (EHR) entry and desk work, in addition to the time spent face to face with patients. Documentation that is completed after hours, or typed while the patient is in the room, reduces the attention available for patient care.

Recent progress in automatic speech recognition (ASR) and natural language processing (NLP) makes it realistic to assist with this task. Large-scale speech models such as Whisper (Radford et al., 2022) can transcribe spoken language robustly, and transformer-based language models (Vaswani et al., 2017; Devlin et al., 2019) can identify and organise information in free text. Together, these technologies suggest a pipeline in which a recorded consultation is converted to text and then to a draft SOAP note that the clinician only needs to review.

This report presents **Scribe AI**, an AI-assisted clinical documentation system that follows this pipeline: voice, to transcript, to SOAP note.

1.2 PROBLEM STATEMENT

Three related problems motivate the project:

- **Time-consuming charting.** Clinicians must document consultations while balancing direct patient care and other clinical responsibilities.
- **Unstructured information.** Spoken information can be lengthy, repetitive, or difficult to convert consistently into a standard clinical format.
- **Documentation burden.** Manual entry can contribute to delays, inconsistent notes, and reduced time available for patient interaction.

1.3 OBJECTIVES

The main objective of the project is to develop an AI-assisted system that converts consultation audio into editable, structured SOAP notes for clinician review. The specific objectives are:

- To allow a doctor to record or upload consultation audio through a web interface.
- To convert the audio into a written transcript using speech recognition.
- To apply NLP to identify symptoms, findings, assessments and treatment plans in the transcript.
- To organise the extracted information into the Subjective, Objective, Assessment and Plan sections of a SOAP note.
- To let the clinician review, edit and approve the draft before it is saved or exported.
- To define an evaluation plan covering transcription quality, generation latency, note completeness and review effort.

1.4 SCOPE AND LIMITATIONS

The scope of Scribe AI is documentation support. The system accepts recorded doctor–patient conversations and produces a draft note. It does not diagnose, prescribe or make independent clinical decisions, and every generated note remains a draft until a qualified clinician has reviewed and approved it.

Performance figures such as transcription accuracy or time saved are not claimed in this report unless they have been measured. Where measured evidence is required, the report marks the location with a highlighted placeholder so that results from actual tests can be added.

1.5 ORGANISATION OF THE REPORT

Chapter 2 reviews the literature relevant to speech-based clinical documentation. Chapter 3 describes the proposed system and its architecture. Chapter 4 describes each module in detail. Chapter 5 covers the implementation approach, experimental setup and results. Chapter 6 concludes the report and outlines future work, and Chapter 7 lists the references.

CHAPTER 2
LITERATURE REVIEW

This chapter reviews four themes that underpin the project: the burden of clinical documentation, automatic speech recognition for medical speech, clinical NLP for structuring free text, and the need for human oversight of AI-generated clinical content.

2.1 CLINICAL DOCUMENTATION BURDEN

Sinsky et al. (2016) studied how physicians in ambulatory practice allocate their time and reported that EHR and desk work occupied a substantial portion of the working day, exceeding the time spent in direct clinical face time. This finding is a major reason for interest in tools that reduce manual note entry.

Ambient AI scribes are one such class of tools. They listen to the clinical encounter and draft documentation automatically. Tierney et al. (2024) described the early use of ambient AI scribes in a large healthcare organisation and discussed their potential to alleviate the burden of clinical documentation, while noting that evaluation of time savings, note quality and clinician experience is needed. For Scribe AI, this means that claimed benefits must be backed by measured time saved and clinician workload.

2.2 THE SOAP NOTE FORMAT

The SOAP note is a standard framework in which the Subjective section records the patient's reported symptoms and history, the Objective section records measurable and observable findings, the Assessment section records the clinician's interpretation, and the Plan section records treatment, investigations and follow-up (Podder, Lew and Ghassemzadeh, 2023). Because the four sections have clearly different purposes, the task of

generating a SOAP note can be treated as classifying and organising information from a conversation into four target categories.

2.3 AUTOMATIC SPEECH RECOGNITION

Whisper is a speech recognition model trained on a large and diverse collection of weakly supervised audio, reported to generalise well across accents, background noise and technical language without dataset-specific fine-tuning (Radford et al., 2022). This makes it a suitable candidate for transcribing consultation audio. However, medical speech contains specialised vocabulary, accents, interruptions and overlapping speakers, so word error rate must be measured on representative audio rather than assumed. Cloud speech-to-text services provide an alternative, with trade-offs in cost and in the handling of sensitive audio.

2.4 CLINICAL NLP AND LANGUAGE MODELS

Information extraction from free text relies on techniques such as named-entity recognition and text classification. The spaCy library provides industrial-strength NLP pipelines built on convolutional neural networks and incremental parsing (Honnibal and Montani, 2017), and can be extended with custom entity rules. Transformer-based models, introduced by Vaswani et al. (2017) and popularised for language understanding by BERT (Devlin et al., 2019), learn contextual representations that support tasks such as entity extraction and sentence classification. Large language models can also be prompted to restructure a transcript directly into note sections.

A known risk of generative models is the production of plausible but unsupported details, often called hallucinations. For clinical documentation, this makes it necessary to check generated notes for completeness, omissions and hallucinated content, and to keep the original transcript visible to the reviewer.

2.5 HUMAN OVERSIGHT

Clinical decision support and documentation tools affect patient records, so errors and uncertainty must be handled carefully. A human-in-the-loop design, in which the clinician reviews, edits and approves the output, is therefore a basic safeguard. Scribe AI adopts this principle: generated notes are drafts, and only a clinician's approval makes them part of the record.

2.6 SUMMARY AND RESEARCH GAP

Table 2.1 summarises the themes and what each implies for Scribe AI.

Theme	Key finding / relevance	Implication for Scribe AI
Clinical documentation burden	Ambient documentation tools aim to reduce time spent on manual note entry and support clinician workflow.	Evaluate time saved and clinician workload.
Automatic speech recognition	Medical speech can include specialised vocabulary, accents, interruptions and background noise.	Measure word error rate on representative audio.
Clinical NLP	Entity extraction and language models can help map free text into structured clinical concepts.	Check completeness, omissions and hallucinated details.
Human oversight	Clinical decision support requires review and careful handling of errors and uncertainty.	Keep the clinician in control of final approval.

Table 2.1: Literature themes and implications for Scribe AI

The reviewed work shows that each building block (speech recognition, NLP and structured note formats) is mature enough to combine. The gap that Scribe AI addresses is a complete, reviewable workflow that connects these blocks end to end, keeps the transcript visible beside the draft note, and defines how the result will be measured, rather than assuming accuracy or speed.

CHAPTER 3
PROPOSED SYSTEM

Scribe AI is a human-in-the-loop pipeline for clinical note generation. The doctor records or uploads consultation audio through a web application. A backend API sends the audio to a speech-to-text component, passes the transcript to an NLP component that extracts clinically relevant information, and assembles the result into a SOAP note. The note is returned to the doctor for review and editing, and only approved notes are stored or exported.

3.1 ARCHITECTURE DIAGRAM

Figure 3.1 shows the architecture of the proposed system. It is organised into a frontend, a backend API server, an AI/NLP module, a database, and external services or models.

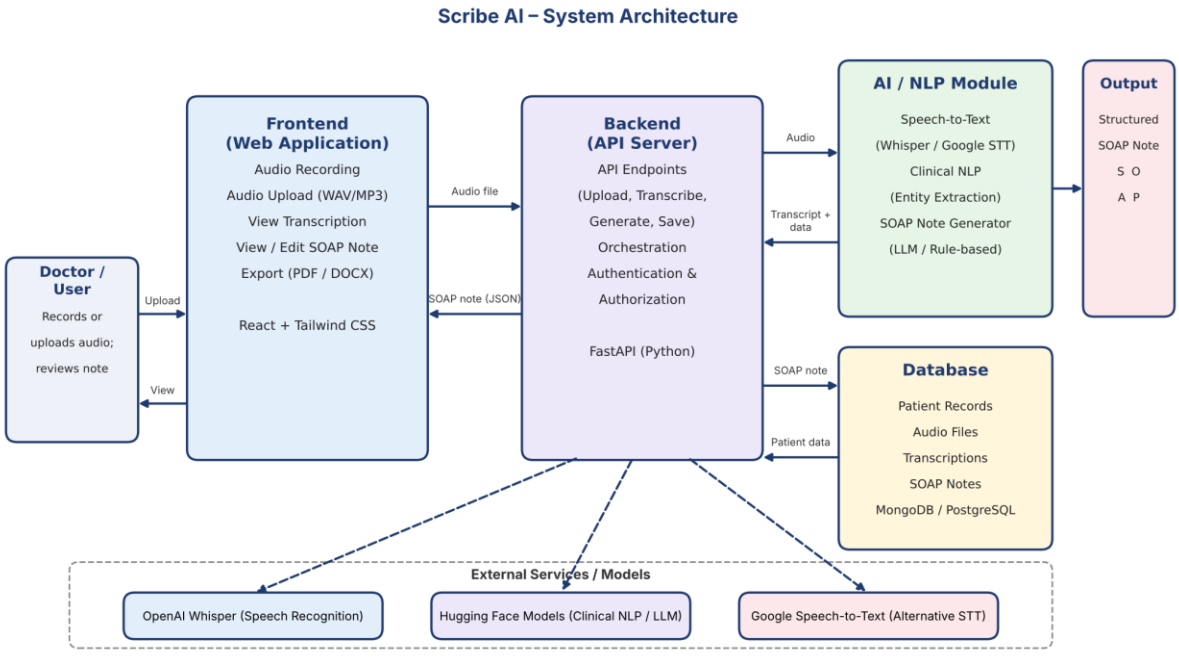


Figure 3.1: System architecture of Scribe AI

Frontend (web application). Built with React and Tailwind CSS, it provides real-time audio recording, audio upload (WAV/MP3), transcript viewing, SOAP note viewing and editing, and export to PDF or document formats.

Backend (API server). Built with Python and FastAPI, it exposes the API endpoints for upload, transcription, note generation and saving. It also handles orchestration of the workflow and authentication and authorisation of users.

AI / NLP module. It contains three stages: speech-to-text (Whisper or Google Speech-to-Text), clinical NLP for entity extraction and classification, and a SOAP note generator that may be LLM-based or rule-based. Its output is a structured SOAP note with Subjective, Objective, Assessment and Plan sections.

Database. MongoDB or PostgreSQL stores patient records, audio files, transcriptions and SOAP notes, and supports retrieval of patient data.

External services and models. The backend may call OpenAI Whisper for speech recognition, Hugging Face models for clinical NLP or language modelling, and Google Speech-to-Text as an alternative speech-recognition service.

3.2 SYSTEM WORKFLOW

The pipeline has five steps, as summarised below.

1. **Capture.** The doctor records or uploads the consultation audio.
2. **Transcribe.** AI converts the audio into a written transcript.
3. **Extract.** NLP identifies relevant symptoms, medical findings, assessments and treatment plans.
4. **Structure.** The extracted information is organised into Subjective, Objective, Assessment and Plan sections.
5. **Review and save.** The doctor reviews, edits and approves the generated notes before saving or exporting them.

The key safeguard is that AI-generated notes remain drafts until reviewed by a qualified clinician.

3.3 TECHNOLOGY STACK

Table 3.1 lists the candidate technologies for each layer. One option per layer is selected for implementation and validated against privacy, deployment, cost and accuracy requirements.

Layer	Technology	Purpose
Frontend	React, HTML, CSS, JavaScript, Tailwind CSS	Responsive interface for audio capture, transcript viewing and note editing.
Backend API	Python and FastAPI	REST endpoints to coordinate transcription, NLP processing and persistence.
Speech recognition	OpenAI Whisper, or a speech-recognition API such as Google Speech-to-Text	Convert consultation audio to text.
Clinical NLP	spaCy, a BERT-based model, or a Hugging Face language model	Clinical information extraction and note generation.
Database	PostgreSQL or MongoDB	Store approved notes, transcripts and metadata.
Export	PDF / document export	Provide notes in a shareable format.

Table 3.1: Technology stack

3.4 API DESIGN

The backend exposes REST endpoints that follow the workflow. Table 3.2 shows the proposed endpoints.

Method	Endpoint	Description
POST	/api/audio/upload	Accept a recorded or uploaded audio file and store it.
POST	/api/transcribe	Run speech recognition and return the transcript.
POST	/api/soap/generate	Extract information and return a draft SOAP note as JSON.

Method	Endpoint	Description
PUT	/api/soap/{id}	Save the clinician's edits to the draft.
POST	/api/soap/{id}/approve	Mark the note as reviewed and approved by the clinician.
GET	/api/soap/{id}/export	Return the approved note as a PDF or document.

Table 3.2: Proposed API endpoints

3.5 DESIGN CONSIDERATIONS

Privacy and security. Consultation data is sensitive. The design calls for authentication, encryption, least-privilege access, retention rules and an appropriate compliance review before any real patient data is used.

Scalability. The API and AI-processing components are kept separate so that longer jobs can be queued, services can be deployed independently, and compute and storage can be scaled as usage grows.

Feasibility. The system is built from established web, API, speech-recognition, NLP and database components, so a prototype can be developed incrementally. The workflow (record, transcribe, generate, edit, approve and save) fits an existing clinician review process.

CHAPTER 4
MODULES DESCRIPTION

The system is divided into seven modules. Each module has a defined input, process and output so that it can be developed, tested and replaced independently.

4.1 MODULE 1: AUDIO CAPTURE AND UPLOAD

This module is part of the frontend. It lets the doctor record a consultation in the browser or upload an existing file in WAV or MP3 format. It validates the file type and size, and sends the audio to the backend upload endpoint.

- **Input:** live microphone audio or an audio file.
- **Process:** recording, format validation, upload.
- **Output:** a stored audio file with a reference identifier.

4.2 MODULE 2: SPEECH-TO-TEXT

This module converts the stored audio into a written transcript using Whisper or an alternative speech-recognition API. Audio is pre-processed (for example, converted to a standard sample rate) before transcription. The transcript is saved and displayed to the doctor, so that the source text behind the note is always visible.

- **Input:** audio file.
- **Process:** pre-processing and speech recognition.
- **Output:** transcript text.

4.3 MODULE 3: CLINICAL NLP AND EXTRACTION

This module analyses the transcript to identify clinically relevant content. It extracts symptoms and patient-reported history, measurements and

examination findings, clinician impressions, and treatment or follow-up instructions. It can use rule-based entity recognition with spaCy, a BERT-based model, or an LLM, and it labels each extracted item with the SOAP section it is most likely to belong to.

- **Input:** transcript text.
- **Process:** sentence segmentation, entity extraction and section classification.
- **Output:** labelled clinical information.

4.4 MODULE 4: SOAP NOTE GENERATION

This module assembles the labelled information into the four-part SOAP structure and returns it as JSON for the frontend. Table 4.1 shows what each section contains.

Section	Typical content	Source in transcript
Subjective	Chief complaint, symptoms, duration, relevant history as reported by the patient.	Patient statements and answers to questions.
Objective	Vital signs, examination findings, test results mentioned in the consultation.	Clinician observations and stated measurements.
Assessment	Clinician's impression or working diagnosis, as stated by the clinician.	Clinician's summarising statements.
Plan	Medication, investigations, advice and follow-up.	Clinician's instructions to the patient.

Table 4.1: SOAP sections and their sources

The generator only reorganises what was said in the consultation. It is not designed to introduce diagnoses or recommendations that the clinician did not state.

4.5 MODULE 5: REVIEW AND EDITING

The doctor sees the transcript beside the draft SOAP note, edits any section, and approves the note. Approval is an explicit action recorded with the user and time. Notes that have not been approved are labelled as drafts and are not exported as final records.

4.6 MODULE 6: STORAGE AND EXPORT

After approval, the note, transcript, audio reference and metadata are stored in the database (PostgreSQL or MongoDB). The module also exports the approved note as a PDF or document, and supports retrieval of earlier notes for a patient.

4.7 MODULE 7: AUTHENTICATION AND AUTHORISATION

This module protects the system. Users sign in before using any endpoint, and access is limited by role so that each user can see only what they need. Transport encryption, retention rules and audit of approvals support the handling of sensitive patient data.

CHAPTER 5
IMPLEMENTATION AND RESULTS

This chapter describes the implementation approach, the experimental setup planned for evaluation, and the results. Items that depend on measurements from actual tests are shown as highlighted placeholders and must be completed with real evidence before submission.

5.1 EXPERIMENTAL SETUP

Implementation approach. The system is built incrementally: first the audio capture and upload screen, then the transcription endpoint, then note generation, and finally review, storage and export. The listings below illustrate the core of the pipeline.

```
# Illustrative: transcription step using the openai-whisper package
import whisper
model = whisper.load_model("base")
result = model.transcribe("consultation.wav")
transcript = result["text"]

# Illustrative: FastAPI endpoint that returns a draft SOAP note
from fastapi import FastAPI
app = FastAPI()


@app.post("/api/soap/generate")
def generate(payload: dict):
    entities = extract_entities(payload["transcript"])
    return build_soap_note(entities)  # {subjective, objective,
assessment, plan}
```

Environment. The prototype environment is described in Table 5.1.

Item	Details
Hardware	Standard laptop or desktop computer, minimum 8 GB RAM and a multi-core CPU with a microphone. A GPU is optional and only speeds up transcription.
Operating system	Windows 10/11 or Ubuntu Linux; the web client runs in any modern browser (Chrome, Edge or Firefox).
Frontend	React with HTML, CSS, JavaScript and Tailwind CSS; Node.js for development tooling.

Item	Details
Backend	Python 3.10 or later with the FastAPI framework, served with Uvicorn.
Speech recognition	OpenAI Whisper (base model, as in the listing above); Google Speech-to-Text as the alternative service.
NLP / note generation	spaCy for entity extraction and sentence processing, with a BERT-based or Hugging Face language model for classifying and generating SOAP sections.
Database	PostgreSQL or MongoDB for patient records, audio references, transcripts and approved notes.

Table 5.1: Experimental environment

Test data. Evaluation uses varied, consented recordings, or simulated doctor–patient conversations, covering different speakers, accents and audio quality. No real patient data is used without consent and an appropriate compliance review.

Evaluation metrics. Table 5.2 lists the metrics used to evaluate the system.

Metric	Definition	Measured on
Word error rate (WER)	(Substitutions + Deletions + Insertions) divided by the number of words in the reference transcript.	Transcripts compared with manual reference.
Generation latency	Time from submitting audio to receiving the draft note.	Each test recording.
Note completeness	Share of clinically relevant items in the reference note that appear in the generated note.	Generated notes compared with clinician reference.
Unsupported content	Number of details in the note that are not supported by the transcript.	Generated notes.
Review effort	Time and number of edits a clinician needs to approve the note.	Clinician review sessions.

Table 5.2: Evaluation metrics

5.2 RESULTS

Application screens. Figures 5.1 to 5.3 show the main screens of the application.

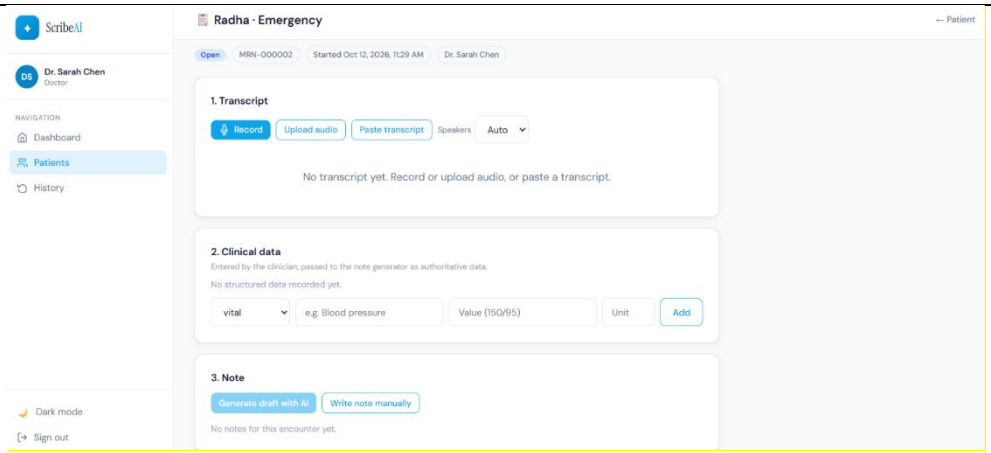


Figure 5.1: Audio Upload Screen

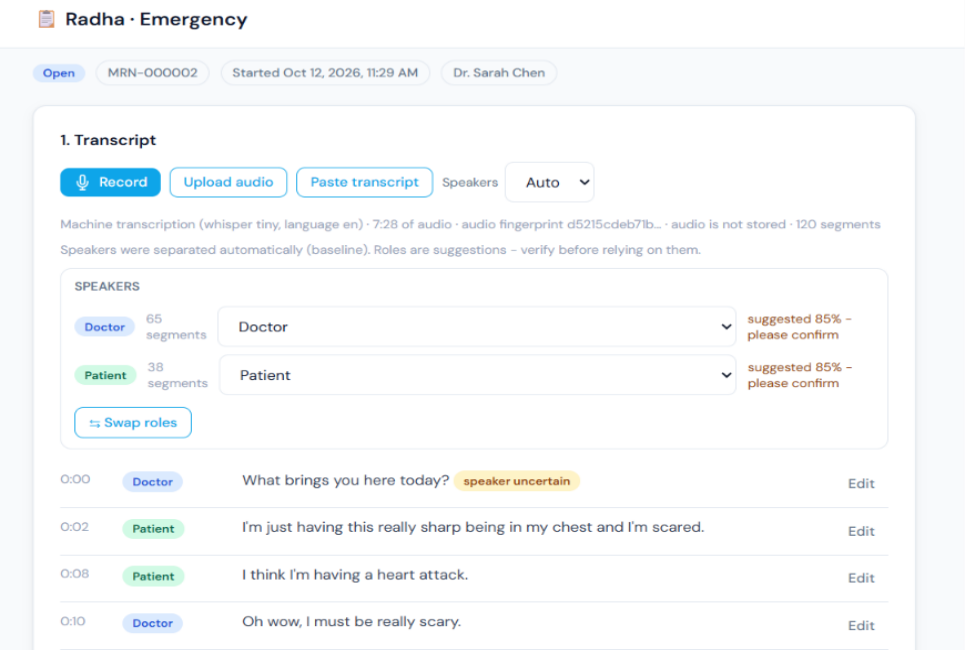


Figure 5.2: Transcription / Processing

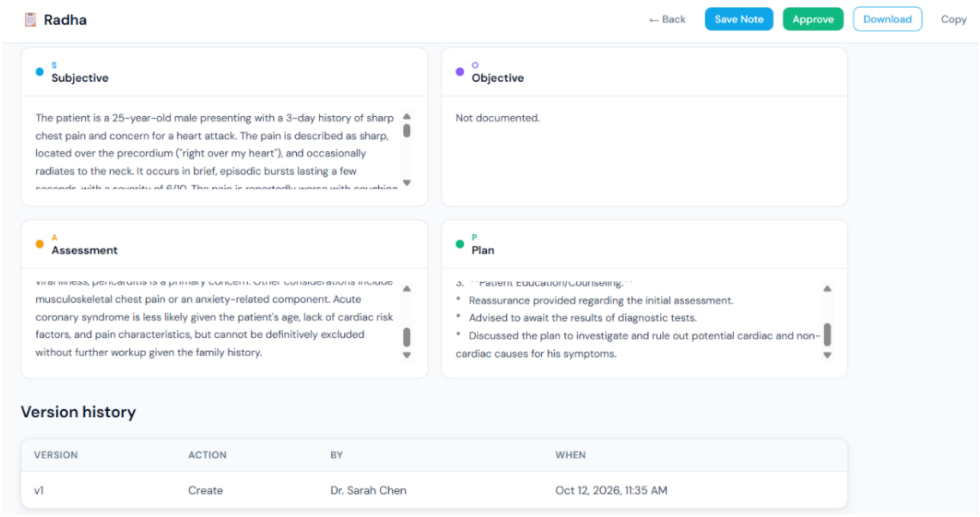


Figure 5.3: SOAP Note Generation

Illustrative example. Table 5.3 shows how a short consultation is expected to map to a SOAP note. The example is synthetic and was written only to illustrate the structure; it is not an output measured from the system.

Item	Content
Transcript excerpt (synthetic)	Patient: I have had a sore throat and fever for two days. Doctor: Your temperature is 100.4 degrees Fahrenheit and the throat looks red. It looks like a throat infection. Take rest, drink fluids and take paracetamol as advised. Come back if it does not improve in three days.
Subjective	Sore throat and fever for two days.
Objective	Temperature 100.4 °F; throat appears red.
Assessment	Throat infection, as stated by the clinician (draft: clinician to confirm).
Plan	Rest, fluids, paracetamol as advised; review if not improved in three days.

Table 5.3: Synthetic example of transcript-to-SOAP mapping

5.3 DISCUSSION

The design reduces the main risks identified in the literature. Audio quality and accents can affect transcription, so testing on varied recordings is planned. Clinical terms may be misrecognised, so the transcript stays visible and the clinician can edit every section. Sensitive data is protected through authentication, encryption, least-privilege access and retention rules. Finally, because performance claims must be validated, the report leaves measured values to be filled in from actual tests instead of assuming them.

CHAPTER 6
CONCLUSION AND FUTURE WORK

6.1 CONCLUSION

This project proposed and described Scribe AI, a clinical documentation assistant that converts consultation audio into structured SOAP notes for clinician review. The system combines a web interface, a FastAPI backend, speech recognition, NLP-based information extraction and persistent storage in a five-step pipeline: capture, transcribe, extract, structure, and review and save. Its central design choice is that AI-generated notes remain drafts until a qualified clinician reviews and approves them. The system is meant to support documentation efficiency and not to replace clinical judgement or make diagnoses independently.

The report defined the architecture, the module structure, an API design and an evaluation plan covering transcription quality, latency, completeness, unsupported content and review effort.

6.2 FUTURE WORK

The following extensions are proposed:

- Complete the measured evaluation on a larger set of consented recordings and report the results.
- Support multiple languages and regional accents, and code-mixed speech.
- Add speaker separation to distinguish doctor and patient automatically.
- Stream audio for real-time transcription during the consultation.
- Integrate with EHR systems through standard interfaces.
- Suggest standard medical codes for clinician confirmation.

- Use clinician edits as feedback to improve note quality over time.
- Evaluate on-premise deployment to keep audio and notes within the hospital network.

CHAPTER 7

REFERENCES

1. Devlin, J., Chang, M.-W., Lee, K. and Toutanova, K. (2019) 'BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding', Proceedings of NAACL-HLT 2019, Minneapolis, pp.4171–4186.
2. FastAPI (n.d.) FastAPI Documentation. Available at: https://fastapi.tiangolo.com/
3. Honnibal, M. and Montani, I. (2017) 'spaCy 2: Natural language understanding with Bloom embeddings, convolutional neural networks and incremental parsing'. Available at: https://spacy.io/
4. Podder, V., Lew, V. and Ghassemzadeh, S. (2023) 'SOAP Notes', StatPearls. Treasure Island (FL): StatPearls Publishing.
5. PostgreSQL Global Development Group (n.d.) PostgreSQL Documentation. Available at: https://www.postgresql.org/docs/
6. Radford, A., Kim, J.W., Xu, T., Brockman, G., McLeavey, C. and Sutskever, I. (2022) 'Robust Speech Recognition via Large-Scale Weak Supervision' (Whisper), arXiv:2212.04356.
7. React (n.d.) React Documentation. Available at: https://react.dev/ .
8. Sinsky, C., Colligan, L., Li, L., Prgomet, M., Reynolds, S., Goeders, L., Westbrook, J., Tutty, M. and Blike, G. (2016) 'Allocation of Physician Time in Ambulatory Practice: A Time and Motion Study in 4 Specialties', Annals of Internal Medicine, Vol.165, No.11, pp.753–760.
9. Tierney, A.A. et al. (2024) 'Ambient Artificial Intelligence Scribes to Alleviate the Burden of Clinical Documentation', NEJM Catalyst Innovations in Care Delivery, Vol.5, No.3.
10. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, Ł. and Polosukhin, I. (2017) 'Attention Is All You

Need', Advances in Neural Information Processing Systems, Vol.30, pp.5998–6008.